

L5

LexFive

ABOGADOS

Resumen del Proyecto

Guía de referencia para replicarlo en otras plataformas

Qué se construyó, con qué tecnologías, cómo está organizado y cómo recrearlo si en otra ocasión se usa otra plataforma o lenguaje de programación.

Versión del manual: junio de 2026

Sitio: lexfive.netlify.app

Contenido

1. Qué es LexFive	2
2. Tecnologías utilizadas (y por qué)	2
3. Estructura del proyecto	2
4. Funcionalidades construidas	3
5. Modelo de datos (resumen)	4
6. Cómo se pone en marcha (una sola vez)	4
7. Guía para replicar el sistema en otra plataforma	5
8. Datos clave del proyecto actual	6

1. Qué es LexFive

LexFive es el sistema digital completo del bufete de abogados LexFive (El Alto, Bolivia). Se compone de dos partes que comparten la misma base de datos:

- Sitio web público: la cara del bufete hacia los clientes (presentación, áreas de práctica, casos de éxito, blog, preguntas frecuentes y formulario de contacto).
- Panel de gestión privado (en /sistema): donde el personal administra procesos judiciales, clientes, consultas, modelos de memoriales, blog, testimonios, usuarios, credenciales y auditoría.

El objetivo del proyecto fue: una web profesional más un sistema de gestión legal, barato de mantener, sin servidor propio y fácil de desplegar.

2. Tecnologías utilizadas (y por qué)

- Frontend: HTML5, CSS3 y JavaScript puro (sin frameworks). Rápido, ligero, sin dependencias ni compilación; fácil de desplegar y mantener.
- Backend y base de datos: Supabase (PostgreSQL gestionado). Aporta base de datos, autenticación y almacenamiento de archivos sin programar un servidor.
- Autenticación: Supabase Auth (correo + contraseña). Login, registro y recuperación de contraseña listos para usar.
- Seguridad de datos: Row Level Security (RLS) de PostgreSQL. Las reglas de acceso por rol viven en la base de datos, no en el navegador.
- Almacenamiento de archivos: Supabase Storage (bucket «documentos»), para memoriales, anexos y archivos de actuaciones.
- Cliente de Supabase: la librería supabase-js cargada por CDN (esm.sh), sin npm ni build.
- Envío de correos del formulario: Web3Forms, servicio gratuito para recibir el formulario de contacto sin backend.
- Hosting: Netlify, conectado a GitHub (despliegue automático al hacer push a main).
- Generación de documentos: scripts en Python que generan manuales, guías y credenciales en formatos .md, .docx y .pdf.

Idea clave

Idea clave del proyecto: todo el «backend» lo provee Supabase. El frontend es estático y la seguridad real la imponen las reglas RLS de la base de datos, no el JavaScript del navegador.

3. Estructura del proyecto

Las páginas públicas están en la raíz; el panel privado, en la carpeta /sistema; los scripts de base de datos, en /db; y los generadores de documentos, en /scripts.

Web pública (raíz)

- index.html — inicio, áreas, nosotros, equipo y contacto.

- casos.html — casos de éxito.
- blog.html — blog jurídico (lee los artículos publicados desde Supabase).
- faq.html — preguntas frecuentes (acordeón).
- verificar.html — verificación pública de credenciales del bufete.
- aviso-privacidad.html y terminos.html — páginas legales.
- css/styles.css — estilos, paleta de colores, temas y diseño responsive.
- js/main.js — interactividad de la web (menú, FAQ, formulario).

Panel de gestión (carpeta /sistema)

- login.html — pantalla de acceso y registro.
- index.html — el panel (dashboard).
- css/panel.css — estilos del panel.
- js/config.js — URL y clave pública de Supabase, roles, abogados y materias.
- js/supabase.js — inicializa el cliente de Supabase.
- js/auth.js — sesión, perfil, roles, permisos y auditoría.
- js/app.js — lógica del panel (procesos, clientes, blog, usuarios, etc.).

Base de datos (carpeta /db)

- schema.sql — tablas base, RLS y almacenamiento.
- 02_portal_clientes.sql — rol «cliente» y aislamiento de datos.
- 03_blog_alertas_testimonios.sql — blog, alertas y testimonios.
- 04_modelos_nurej.sql — biblioteca de modelos y campo NUREJ.
- 05_multiples_abogados.sql — varios abogados/procuradores por proceso.
- 06_consultas.sql — bandeja de consultas (formulario de contacto).
- 07_sync_clientes.sql — crea la ficha de cliente al registrarse.
- 08_categorias.sql — áreas del derecho dinámicas.
- 09_actuaciones_archivos.sql — adjuntar archivos a cada actuación.

4. Funcionalidades construidas

Sitio web público

- Página de inicio con áreas de práctica, equipo, testimonios e indicadores.
- Casos de éxito, blog jurídico y preguntas frecuentes.
- Formulario de contacto que guarda cada mensaje en la base de datos (bandeja de Consultas) y, opcionalmente, envía copia por correo (Web3Forms).
- Botón flotante de WhatsApp con menú de los cinco abogados.
- Páginas legales y página de verificación de credenciales.
- Cuatro temas de color con acento dorado y diseño accesible y responsive.

Panel de gestión privado

- Dashboard con métricas (procesos totales/activos, audiencias próximas, consultas nuevas).

- Procesos/casos: crear, editar, asignar varios abogados y procuradores, vincular cliente, estados, NUREJ, próxima audiencia e historial de actuaciones con archivos.
- Clientes: cartera con documento, contacto y vinculación por correo.
- Bandeja de consultas con estados (Nueva/Atendida/Archivada) y respuesta por WhatsApp o correo.
- Modelos de memoriales: biblioteca de plantillas por área del derecho.
- Blog: redacción de artículos (borrador/publicado) que aparecen en la web pública.
- Testimonios: moderación (aprobar/rechazar) antes de publicarse.
- Usuarios (solo admin): cambiar roles, activar/desactivar cuentas.
- Categorías/áreas del derecho (solo admin): crear, renombrar y eliminar.
- Auditoría (solo admin): bitácora de acciones importantes.
- Credenciales: generador de carnet imprimible del bufete.
- Portal del cliente: vista reducida donde el cliente ve solo sus procesos y deja su opinión.
- Autoguardado de borradores y cierre de sesión por inactividad.

5. Modelo de datos (resumen)

Tablas principales en PostgreSQL (Supabase):

- **profiles** — usuarios del sistema, vinculados a `auth.users`, con rol (admin/procurador/abogado/cliente) y estado activo.
- **clientes** — cartera de clientes (vinculados por correo).
- **procesos** — casos judiciales/administrativos (carátula, número/NUREJ, materia, estado, audiencia, etc.).
- **actuaciones** — historial de pasos de cada proceso (con archivos adjuntos).
- **documentos** — metadatos de memoriales/archivos (el archivo físico va en Storage).
- **articulos** — artículos del blog (borrador/publicado).
- **auditoria** — bitácora de acciones.
- Más tablas para consultas, testimonios y categorías, añadidas por los scripts 02 a 09.

Seguridad

Seguridad (RLS): cada tabla tiene políticas que definen quién puede ver, crear, editar o borrar. Por ejemplo, todos ven los procesos pero solo el admin elimina; un cliente solo ve sus propios procesos; la auditoría solo la ve el admin. Funciones auxiliares: `is_admin()` y `current_rol()`.

6. Cómo se pone en marcha (una sola vez)

1. Crear un proyecto en Supabase.
2. En el SQL Editor, ejecutar en orden los scripts de db/: primero `schema.sql`, luego del 02 al 09.
3. En Authentication ? Providers ? Email, configurar el login por correo.
4. Crear el primer usuario y convertirlo en admin con: `update public.profiles set rol = 'admin' where email = 'tu-correo';`
5. Poner la URL y la clave pública de Supabase en `sistema/js/config.js`.

6. (Opcional) Poner la clave de Web3Forms en el formulario de index.html.
7. Desplegar en Netlify conectando el repositorio de GitHub (push a main = publica solo).

Más detalle

El detalle completo está en sistema/CONFIGURACION.md y en el Manual del Sistema.

7. Guía para replicar el sistema en otra plataforma

Si en el futuro se reconstruye con otro stack (por ejemplo React + Node, Django, Laravel, Firebase, etc.), estos son los componentes que hay que cubrir, independientemente del lenguaje.

Equivalencias por capa (lo usado aquí y sus alternativas)

- Base de datos relacional — aquí: Supabase (PostgreSQL). Alternativas: Firebase/Firestore, MySQL, MongoDB, PlanetScale, Neon.
- Autenticación — aquí: Supabase Auth. Alternativas: Firebase Auth, Auth0, Clerk, NextAuth, Devise (Rails), auth de Django.
- Seguridad por rol — aquí: RLS de PostgreSQL. Alternativas: middleware/guards en el backend (guards de NestJS, permissions de Django, policies de Laravel).
- Almacenamiento de archivos — aquí: Supabase Storage. Alternativas: AWS S3, Firebase Storage, Cloudinary.
- Frontend — aquí: HTML/CSS/JS puro. Alternativas: React, Vue, Angular, Svelte, Next.js.
- Recepción de formularios — aquí: Web3Forms. Alternativas: Formspree, EmailJS, o un endpoint propio con Nodemailer/SendGrid.
- Hosting — aquí: Netlify. Alternativas: Vercel, GitHub Pages, Render, Cloudflare Pages o un VPS.

Checklist funcional para no olvidar nada

- Roles y permisos: admin, abogado, procurador, cliente — con quién puede borrar.
- Gestión de procesos con asignación múltiple de abogados/procuradores.
- Historial de actuaciones con archivos adjuntos por paso.
- Clientes vinculados por correo y portal del cliente (ve solo lo suyo).
- Bandeja de consultas alimentada por el formulario web.
- Biblioteca de modelos por área del derecho.
- Blog editable desde el panel que se publica en la web.
- Testimonios con moderación.
- Categorías dinámicas (áreas del derecho).
- Auditoría de acciones.
- Generador de credenciales imprimibles.
- Autoguardado de borradores y cierre de sesión por inactividad.

Lecciones aprendidas y recomendaciones

- Empezar por el modelo de datos y los roles: todo lo demás (pantallas, permisos) se deriva de ahí. Tener clara la matriz de permisos antes de programar evita rehacer trabajo.

- Mantener la seguridad en el backend o la base de datos, nunca solo en el navegador. Aquí se logró con RLS; en otro stack, con guards/políticas del lado del servidor.
- Separar claves públicas de claves secretas. En config.js solo va la clave pública; la service_role y las contraseñas jamás se exponen en el frontend.
- Un frontend estático más un backend gestionado (BaaS) abarata y simplifica el mantenimiento de proyectos pequeños y medianos: no hay servidor que administrar.
- El despliegue continuo (push a main que publica solo) ahorra mucho tiempo.
- Documentar desde el inicio (manual y guía de configuración) facilita el traspaso y el soporte.

8. Datos clave del proyecto actual

- Sitio: `lexfive.netlify.app`.
- Frontend: HTML/CSS/JS puro (alrededor de 3.700 líneas entre `app.js`, `main.js` y `styles.css`).
- Base de datos: Supabase (PostgreSQL), 9 scripts SQL versionados en `db/`.
- Despliegue: Netlify conectado a GitHub (rama main).
- Repositorio: `carloscartagena/LexFive`.

Versión

Este resumen describe el proyecto a la fecha indicada en la portada. Si el sistema cambia, vuelva a generar el documento con `scripts/generar_resumen.py`.